



รายงานการโครงการรายวิชา CN230
เรื่อง

ระบบเว็บแอปพลิเคชันสำหรับการบริหารจัดการและจับคู่สมาชิกกลุ่ม

ชื่อผู้ทำโครงการ นาย ชัยวัฒน์ ท้วมวิล

บทคัดย่อ

ระบบเว็บแอปพลิเคชันสำหรับการบริหารจัดการและจับคู่สมาชิกกลุ่ม รายวิชา: CN230 Database

โครงการนี้มีวัตถุประสงค์เพื่อแก้ไขปัญหาความล่าช้าและความคลาดเคลื่อนในการสื่อสารเมื่อต้องจัดตั้งกลุ่มทำงานในบริบทของการเรียนการสอน ซึ่งมักเกิดปัญหาความสับสนในการหาบุคคลที่มีความสนใจตรงกัน หรือการค้นหากลุ่มที่ยังมีที่ว่างเหลืออยู่ ผู้จัดทำจึงได้พัฒนาระบบเว็บแอปพลิเคชันเพื่อเป็นตัวกลางในการเชื่อมโยงนักศึกษาที่ต้องการกลุ่มและกลุ่มที่ต้องการสมาชิกเข้าด้วยกันอย่างมีประสิทธิภาพ

ในด้านการพัฒนาระบบ ได้ประยุกต์ใช้สถาปัตยกรรมแบบ Full-stack โดยใช้ React ในการพัฒนาส่วนติดต่อผู้ใช้งาน (Frontend) และใช้ Node.js (Express) เป็นส่วนประมวลผลหลังบ้าน (Backend) สำหรับจัดการ Logic ของระบบ ในส่วนของการจัดการข้อมูล ได้นำความรู้จากรายวิชา CN230 Database Systems มาประยุกต์ใช้ในการออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ด้วย MySQL โดยเน้นการจัดเก็บข้อมูลสมาชิก กลุ่ม และสถานะการตอบรับอย่างเป็นระบบ เพื่อให้ข้อมูลมีความถูกต้อง (Data Integrity) และสามารถเรียกใช้งานได้อย่างรวดเร็ว

คำสำคัญ: ระบบฐานข้อมูล, Web Application, React, Node.js, MySQL, การจัดกลุ่มทำงาน

คำนำ

ระบบเว็บแอปพลิเคชันสำหรับการบริหารจัดการและจับคู่สมาชิกกลุ่ม รายวิชา: CN230 Database

โครงงานนี้มีวัตถุประสงค์เพื่อแก้ไขปัญหาความล่าช้าและความคลาดเคลื่อนในการสื่อสารเมื่อต้องจัดตั้งกลุ่มทำงาน
ในบริบทของการเรียนการสอน ซึ่งมักเกิดปัญหาความสับสนในการหาบุคคลที่มีความสนใจตรงกัน หรือการ
ค้นหากลุ่มที่ยังมีที่ว่างเหลืออยู่ ผู้จัดทำจึงได้พัฒนาระบบเว็บแอปพลิเคชันเพื่อเป็นตัวกลางในการเชื่อมโยง
นักศึกษาที่ต้องการกลุ่มและกลุ่มที่ต้องการสมาชิกเข้าด้วยกันอย่างมีประสิทธิภาพ

สารบัญ

บทคัดย่อ	3
สารบัญ	4
บทที่ 1 บทนำ	5
ความเป็นมาและความสำคัญ	
วัตถุประสงค์ของโครงการ	
ขอบเขตโครงการ	
คำจำกัดความที่ใช้	
ประโยชน์ที่คาดว่าจะได้รับ	
บทที่ 2 โปรแกรมที่เกี่ยวข้องในโครงการ	7
บทที่ 3 วิธีดำเนินการโครงการ	9
ประชากรและการสุ่มกลุ่มตัวอย่าง	
การดำเนินงานโครงการ	
บทที่ 4 ผลที่ได้จากโครงการ	20

1. บทนำ

1.1 ความเป็นมาและความสำคัญของปัญหา

ในบริบทของการเรียนการสอนระดับมหาวิทยาลัย การมอบหมายงานกลุ่ม (Group Assignment) เป็นส่วนสำคัญในการส่งเสริมทักษะการทำงานร่วมกัน อย่างไรก็ตาม ปัญหาที่พบบ่อยคือ "ความเสียดทานในการจัดกลุ่ม" (Group Formation Friction) นักศึกษามักประสบความยากลำบากในการหาสมาชิก หรือ การค้นหาว่ากลุ่มใดที่ยังต้องการสมาชิกเพิ่ม ซึ่งกระบวนการเดิมมักทำผ่านการสอบถามส่วนตัวหรือการประกาศในกลุ่มแชทขนาดใหญ่ ทำให้ข้อมูลกระจุกกระจายและตกหล่นได้ง่าย

จากปัญหาดังกล่าว ผู้จัดทำจึงเห็นควรในการพัฒนาระบบฐานข้อมูลและเว็บแอปพลิเคชันเพื่อเป็นสื่อกลางในการบริหารจัดการสมาชิกกลุ่ม โดยนำองค์ความรู้จากรายวิชา **CN230 Database Systems** มาประยุกต์ใช้ในการออกแบบฐานข้อมูลที่มีประสิทธิภาพ เพื่อลดขั้นตอนที่ซ้ำซ้อนและช่วยให้นักศึกษาสามารถจับคู่กลุ่มทำงานได้อย่างรวดเร็วและเป็นระบบมากยิ่งขึ้น

1.2 วัตถุประสงค์ของการวิจัย

1. เพื่อออกแบบและพัฒนาระบบฐานข้อมูลเชิงสัมพันธ์ (Relational Database) สำหรับการจัดเก็บข้อมูลสมาชิกและกลุ่มงาน
2. เพื่อพัฒนาเว็บแอปพลิเคชันที่ช่วยลดปัญหาความเสียดทานในการหากลุ่มทำงานของนักศึกษา
3. เพื่อประยุกต์ใช้ความรู้ด้าน Full-stack Development (React, Node.js และ MySQL) ในการแก้ไขปัญหาที่เกิดขึ้นจริงในสถาบันการศึกษา

1.3 ขอบเขตการวิจัย

1.3.1. ด้านการพัฒนาระบบ:

ใช้ MySQL ในการออกแบบและจัดการระบบฐานข้อมูล

ใช้ Node.js สำหรับการพัฒนาส่วน API และ Backend

ใช้ React, Tailwind css สำหรับการพัฒนาส่วนติดต่อผู้ใช้งาน (Frontend)

1.3.2.ด้านฟังก์ชันการทำงาน:

ระบบสามารถลงทะเบียนเข้าใช้งาน และ เลือกโปรไฟล์ผู้ใช้ได้

ระบบสามารถสร้างกลุ่ม และ กำหนดจำนวนสมาชิกที่ต้องการได้

ผู้ใช้งานสามารถค้นหากลุ่มที่ว่างอยู่และเข้าร่วมกลุ่มได้

ผู้ใช้งานสามารถออกจากกลุ่มได้

ผู้ใช้งานสามารถกำหนดให้งานเสร็จแล้วได้

ผู้ใช้งานสามารถสับค้นประวัติงานที่ทำเสร็จไปแล้ว

1.3.3.กลุ่มเป้าหมาย:

นักศึกษาที่ลงทะเบียนในรายวิชาที่มีการทำงานกลุ่ม

ผู้ที่ต้องการหาเพื่อนร่วมทีมในโปรเจกต์ต่าง ๆ

ผู้ที่ต้องการหาสังคม ที่ชอบงานอดิเรกเดียวกัน

นักศึกษาที่ไม่ได้มาเรียน หรือ ต้องทำงานกลุ่มแบบ remote

นักศึกษาที่ต้องการประวัติค้นหางานกลุ่มในรายวิชาต่างๆ ในปีก่อน

1.3.4 Stakeholders

1. ผู้ค้นหากลุ่ม (Adventurers/Seekers):

- สิ่งที่ต้องทำ: เข้าสู่ระบบ (Login) → เลือกดูบอร์ดภารกิจ (Main Board) → ค้นหาแท็กที่สนใจ (Search by Tag) → ตรวจสอบรายละเอียดสมาชิกที่ว่าง → กดเข้าร่วมกลุ่ม (Join Party)

2. ผู้ต้องการสร้างกลุ่ม (Party Leaders/Quest Givers):

- **สิ่งที่ต้องทำ:** เข้าสู่ระบบ → สร้างประกาศภารกิจ (Post Bounty) → กำหนดชื่อกลุ่มและจำนวนสมาชิกสูงสุด (Max Member) → ให้ข้อมูลการติดต่อ (Contact) → บริหารจัดการการส่งผลงาน และสำเร็จภารกิจ
- 3. **ผู้ใช้งานระบบเกม (RPG System Users):**
 - **สิ่งที่ต้องทำ:** เข้าสู่ระบบ → ไปที่หน้า Bank (ตรวจสอบ/โอนเงิน) → ไปที่หน้า Blacksmith (อัปเกรด/ปรับแต่งอุปกรณ์) เพิ่ม score ของผู้ใช้งาน
- 4. **ผู้ต้องการตรวจสอบข้อมูลย้อนหลัง (Archivers):**
 - **สิ่งที่ต้องทำ:** เข้าสู่ระบบ → ไปที่หน้า Search → เข้าไปดูประวัติการโพสต์หรือการรวมกลุ่มในหน้า Profile

1.4 Stakeholders

Use Case 1: การประกาศหาพรรคพวก (Post Bounty)

- **ชื่อ Use Case:** การสร้างประกาศภารกิจหากลุ่ม
- **ผู้ใช้งานหลัก:** ผู้ต้องการสร้างกลุ่ม (Party Leader)
- **สิ่งที่ทำ (Action):** ผู้ใช้กรอกรายละเอียดภารกิจ (หัวข้อ, รายละเอียด, รางวัล, แท็ก) และระบุจำนวนสมาชิกที่ต้องการลงในฟอร์มหน้า Post Bounty
- **ผลที่เกิดขึ้น (Outcome):**
 1. ข้อมูลถูกบันทึกลงในตาราง post, contact, และ party
 2. ระบบทำการเพิ่มผู้สร้างกลุ่มลงในตาราง member โดยอัตโนมัติในฐานะสมาชิกคนแรก
 3. ประกาศจะไปปรากฏที่หน้า Main Board เพื่อให้ผู้อื่นเห็น

Use Case 2: การเข้าร่วมกลุ่ม (Join Party)

- **ชื่อ Use Case:** การสมัครเข้าร่วมภารกิจ
- **ผู้ใช้งานหลัก:** ผู้ค้นหากลุ่ม (Adventurer)
- **สิ่งที่ทำ (Action):** ผู้ใช้เลือกภารกิจที่สนใจจากหน้า Main Board แล้วกดปุ่ม "Join Party"
- **ผลที่เกิดขึ้น (Outcome):**
 1. ระบบตรวจสอบสถานะกลุ่มในตาราง party ว่ายังไม่เต็ม (Member < Max)
 2. ระบบ UPDATE เพิ่มจำนวนสมาชิกในตาราง party ขึ้น 1 คน

3. ระบบ INSERT ข้อมูลผู้ใช้และ ID ของโพสต์ลงในตาราง member
4. รายการภารกิจนั้นจะหายไปจากหน้า Main Board ของผู้ใช้นั้น (ตาม Logic Query ที่คุณเขียนไว้)

Use Case 3: การตีบวกอุปกรณ์ (Blacksmith Upgrade)

- ชื่อ Use Case: การปรับแต่งอุปกรณ์
- ผู้ใช้งานหลัก: ผู้เล่นทั่วไป
- สิ่งที่ทำ (Action): ผู้ใช้เข้าหน้า Blacksmith เลือกอุปกรณ์จากตาราง gear ที่ต้องการซื้อ
- ผลที่เกิดขึ้น (Outcome):
 1. ระบบตรวจสอบยอดเงินในตาราง bank ว่าเพียงพอหรือไม่
 2. ระบบทำการ UPDATE ค่าสถานะ (Score/Rarity) ในตาราง gear หรือ blacksmith
 3. ระบบหักยอดเงินออกจากตาราง bank

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. ช่วยลดระยะเวลาและขั้นตอนในการจัดตั้งกลุ่มทำงานให้มีความสะดวกรวดเร็วขึ้น
2. ลดปัญหาการไม่มีกลุ่มทำงานหรือการสื่อสารที่คลาดเคลื่อนระหว่างการหาเพื่อนร่วมทีม
3. ได้ระบบฐานข้อมูลที่มีโครงสร้างถูกต้องตามหลักการ Normalization และสามารถนำไปต่อยอดใช้งานได้จริง
4. ผู้จัดทำได้เพิ่มพูนทักษะการแก้ปัญหาด้วยการเขียนโปรแกรมและการจัดการฐานข้อมูลอย่างเป็นระบบ

1.5 คำจำกัดความที่ใช้ในงานวิจัย

- ระบบเว็บแอปพลิเคชัน (Web Application): โปรแกรมประยุกต์ที่ทำงานผ่านเบราว์เซอร์ โดยมีการประมวลผลผ่านเซิร์ฟเวอร์
- ความเสียดทานในการหาทีม (Group Friction): อุปสรรคหรือความยากลำบากในขั้นตอนการสื่อสาร ค้นหา และตอบรับ เพื่อจัดตั้งทีมงาน
- Backend: ส่วนการประมวลผลหลังบ้านที่จัดการ Logic และการเชื่อมต่อฐานข้อมูล (ในที่นี้คือ Node.js)
- Frontend: ส่วนการแสดงผลและโต้ตอบกับผู้ใช้งาน (ในที่นี้คือ React)

Data Integrity: ความถูกต้องและความสอดคล้องของข้อมูลภายในระบบฐานข้อมูลที่ได้รับการออกแบบตามหลักวิชา CN230

บทที่ 2

โปรแกรมและเครื่องมือที่เกี่ยวข้อง

ในการพัฒนาระบบเว็บแอปพลิเคชันหาสมาชิกกลุ่ม ผู้จัดทำได้เลือกใช้เครื่องมือที่รองรับการทำงานในรูปแบบ Full-stack Development และสอดคล้องกับหลักการจัดการฐานข้อมูลในรายวิชา CN230 โดยมีรายละเอียดดังนี้

2.1 Visual Studio Code (VS Code)

เป็นโปรแกรมสำหรับเขียนรหัสโปรแกรม (Code Editor) ที่ได้รับความนิยมสูง เนื่องจากมีความยืดหยุ่นและรองรับส่วนขยาย (Extensions) ที่หลากหลาย

- **บทบาท:** ใช้เป็นเครื่องมือหลักในการเขียน Code ทั้งส่วน Frontend, Backend และการจัดรูปแบบ CSS
- **การติดตั้ง:** ดาวน์โหลดตัวติดตั้งจากเว็บไซต์อย่างเป็นทางการ เลือกเวอร์ชันที่ตรงกับระบบปฏิบัติการ (Windows/macOS) และดำเนินการติดตั้งตามขั้นตอนมาตรฐาน

2.2 Node.js

สภาพแวดล้อมสำหรับการทำงานของ JavaScript บนฝั่ง Server (Runtime Environment)

- **บทบาท:** ใช้สำหรับรัน Backend ที่พัฒนาด้วย Express Framework และใช้จัดการ Library ต่างๆ ผ่าน npm (Node Package Manager) เช่น mysql2 สำหรับเชื่อมต่อฐานข้อมูล
- **การติดตั้ง:** ติดตั้งเวอร์ชัน LTS (Long Term Support) เพื่อความเสถียรของระบบ ตรวจสอบการติดตั้งผ่าน Terminal ด้วยคำสั่ง `node -v` และ `npm -v`

2.3 MySQL Workbench

เครื่องมือบริหารจัดการฐานข้อมูลในรูปแบบ Visual Tool

- **บทบาท:** ใช้ในการออกแบบโครงสร้างฐานข้อมูล (Schema Design), การเขียนคำสั่ง SQL (Query), และการจัดการความสัมพันธ์ของข้อมูล (ER Diagram) ตามหลักการที่เรียนใน CN230
- **การติดตั้ง:** ติดตั้งพร้อมกับ MySQL Community Server โดยใช้ตัวติดตั้ง MySQL Installer เพื่อตั้งค่า Root Password สำหรับการเชื่อมต่อ

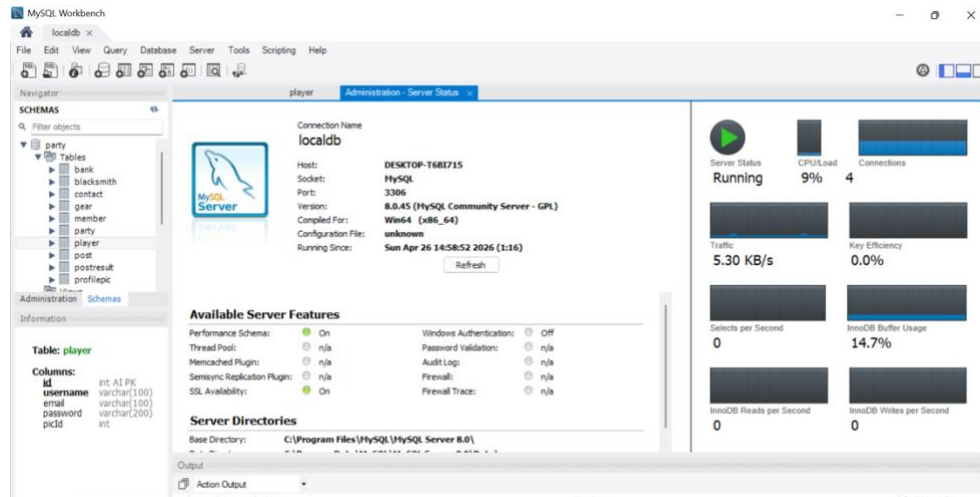
2.4 ส่วนขยาย (Extensions) สำหรับ Visual Studio Code

เพื่อให้การพัฒนาด้วย React และ Node.js มีประสิทธิภาพและลดความผิดพลาด ผู้จัดทำได้ติดตั้งส่วนขยายที่สำคัญดังนี้:

1. **ES7+ React/Redux/React-Native snippets:** ช่วยในการสร้างโครงสร้าง Component ของ React ได้รวดเร็วผ่านการใช้คำสัณย่อ (Shortcuts)
2. **JavaScript (ES6) code snippets:** รองรับไวยากรณ์ JavaScript สมัยใหม่ ทำให้เขียนโค้ดได้กระชับและเป็นระเบียบ
3. **HTML CSS Support:** ช่วยในการเติมคำอัตโนมัติ (Autocomplete) สำหรับคลาส CSS และแท็ก HTML ช่วยให้การตกแต่งหน้าจอกับ Tailwind CSS หรือ CSS ปกติทำได้ง่ายขึ้น
4. **Prettier - Code formatter:** ช่วยจัดรูปแบบโค้ดให้สวยงามและเป็นมาตรฐานสากลโดยอัตโนมัติเมื่อทำการบันทึกไฟล์

2.5 เทคโนโลยีเสริมและส่วนเชื่อมต่อ

- **React:** Library ฝั่ง Frontend ที่ใช้แนวคิด Component-based ในการสร้าง UI ที่ตอบสนองได้รวดเร็ว
- **MySQL2 Driver:** ส่วนเชื่อมต่อระหว่าง Node.js และ MySQL ที่รองรับการทำงานแบบ Promise-based ช่วยให้การดึงข้อมูลจากฐานข้อมูลมีความเสถียร
- ในการทดสอบประสิทธิภาพและความถูกต้องของ API ผู้จัดทำเลือกใช้เครื่องมือ **Insomnia REST Client**



บทที่ 3

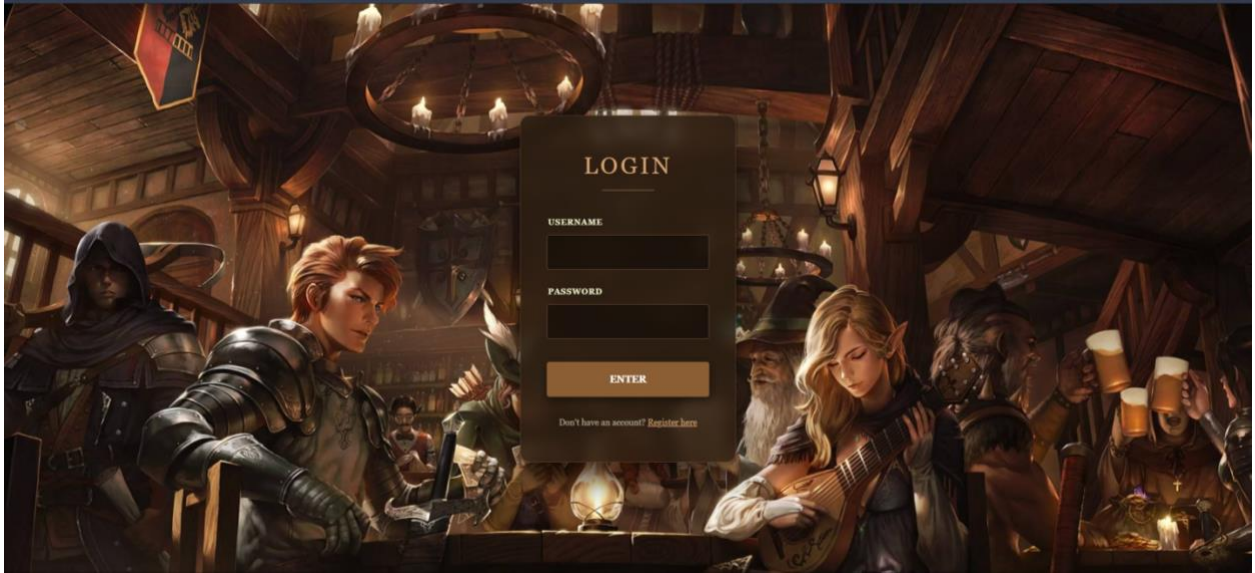
วิธีดำเนินการวิจัย

ในการพัฒนาระบบเว็บแอปพลิเคชันเพื่อการหาสมาชิกกลุ่ม ผู้จัดทำได้ดำเนินการตามขั้นตอนการพัฒนาระบบทางซอฟต์แวร์ โดยแบ่งออกเป็นส่วนของทฤษฎีความต้องการ การออกแบบฐานข้อมูล และการพัฒนาส่วนติดต่อผู้ใช้งาน ดังนี้

3.1 การวิเคราะห์ความต้องการ (Requirements Analysis)

จากการวิเคราะห์ปัญหาความเสียดทานในการหากลุ่ม ได้กำหนดฟังก์ชันการทำงานของระบบโดยใช้ธีม "Fantasy Adventure Guild Board" (กระดานภารกิจแบบแฟนตาซี) เพื่อสร้างประสบการณ์การใช้งานที่น่าสนใจ โดยแบ่งเป็นโมดูลดังนี้:

1. ระบบพิสูจน์ตัวตน: Login และ Register สำหรับเข้าสู่ระบบ
2. ระบบจัดการภารกิจ: Board (Main), Post Bounty และ Search สำหรับการหากลุ่ม/หาภารกิจ
3. ระบบจัดการกลุ่ม: Party และ Submit Result สำหรับการบริหารจัดการสมาชิกและการส่งงาน
4. ระบบเสริม: Bank (จัดการแท้ม/รางวัล), Profile (ข้อมูลนักผจญภัย) และ Blacksmith (การตั้งค่า/เครื่องมือ)

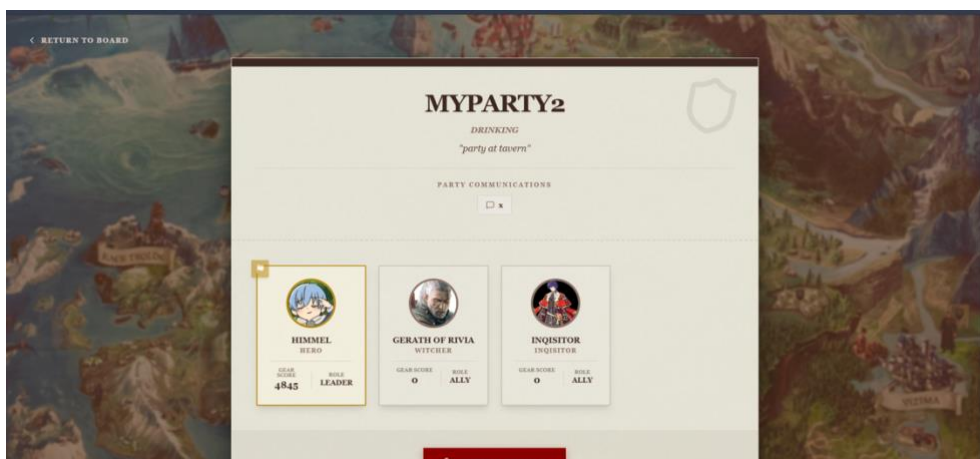


3.2 การออกแบบส่วนติดต่อผู้ใช้งาน (UI/UX Design)

ผู้จัดทำได้กำหนดแนวทางการออกแบบในสไตล์ **Fantasy Wooden & Parchment** เพื่อให้ระบบมีความแตกต่างจากเว็บแอปพลิเคชันทั่วไป โดยมีรายละเอียดลำดับการทำงาน (User Flow) ดังนี้:

- **Authentication (Login/Register):** ออกแบบหน้าจอให้เหมือนการลงทะเบียนเข้ากิลด์นักผจญภัย ใช้พื้นหลังลายกระดาษเก่า (Parchment) และปุ่มกดลายไม้
- **The Quest Board (Main):** หน้าหลักที่แสดงรายการ "Bounty" หรือประกาศหากลุ่ม โดยเรียงลำดับตามความสดใหม่ของข้อมูล
- **Party Management:** หน้าจอแสดงรายชื่อสมาชิกในกลุ่ม (Party)

- The Bank & Blacksmith: ส่วนจัดการข้อมูลส่วนตัวและระบบหลังบ้าน โดยใช้ไอคอนที่สื่อถึงร้านค้าในโลกแฟนตาซี

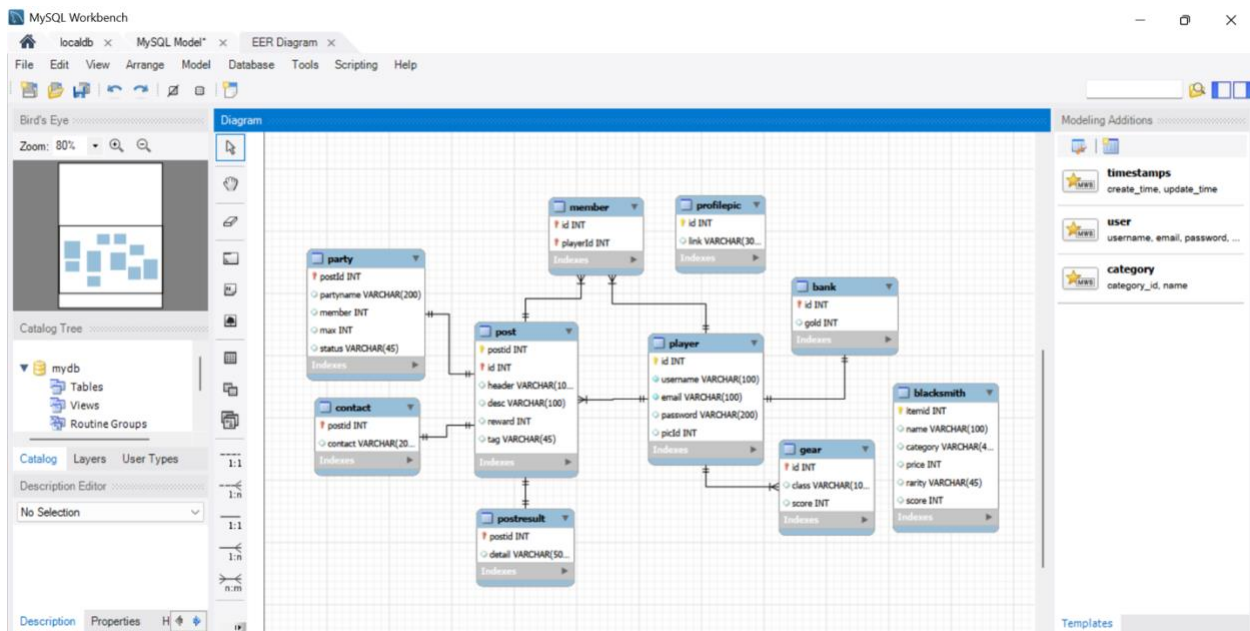


3.3 การออกแบบฐานข้อมูล (Database Design)

จากการประยุกต์ใช้ความรู้ในรายวิชา CN230 ผู้จัดทำได้ออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ (Relational Database) เพื่อรองรับฟังก์ชันข้างต้น ประกอบด้วยตารางหลักดังนี้:

1. **Users Table:** เก็บข้อมูลพื้นฐาน (Username, Password, email, pictureid)
2. **Post / Party Table:** เก็บข้อมูลประกาศหากลุ่ม หัวข้อภารกิจ และจำนวนสมาชิกที่ต้องการ
3. **MemberTable:** เก็บความสัมพันธ์ระหว่าง Users และ Parties (เพื่อระบุว่าใครอยู่กลุ่มไหน และมีสถานะเป็นหัวหน้ากลุ่มหรือสมาชิก)

4. Postresult Table: สำหรับระบบ Bank เพื่อบันทึกประวัติการทำกิจกรรม



การออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ ได้มี foreign key และ primary key ตามแต่ละตารางดังนี้

3.3.1. ตารางกลุ่มผู้เล่นและการแสดงผล

- **player:**
 - Primary Key: id (INT) key หลักของระบบ
- **profile pic**
 - Primary Key: id ของภาพตัวละคร
- **bank:**
 - Primary Key: id (INT) (เป็นทั้ง PK และเป็น FK ที่เชื่อมมาจาก player(id) (1:1)

3.3.2. ตารางกลุ่มภารกิจและการจัดการปาร์ตี้

- **post:** (ตารางหลักของระบบ Bounty Board)
 - Primary Key: id (INT)
 - Foreign Key: id (ภายในตาราง) เชื่อมโยงกลับไปยัง player(id) เพื่อระบุว่าใครเป็นผู้สร้างประกาศ (Owner) (M :1)

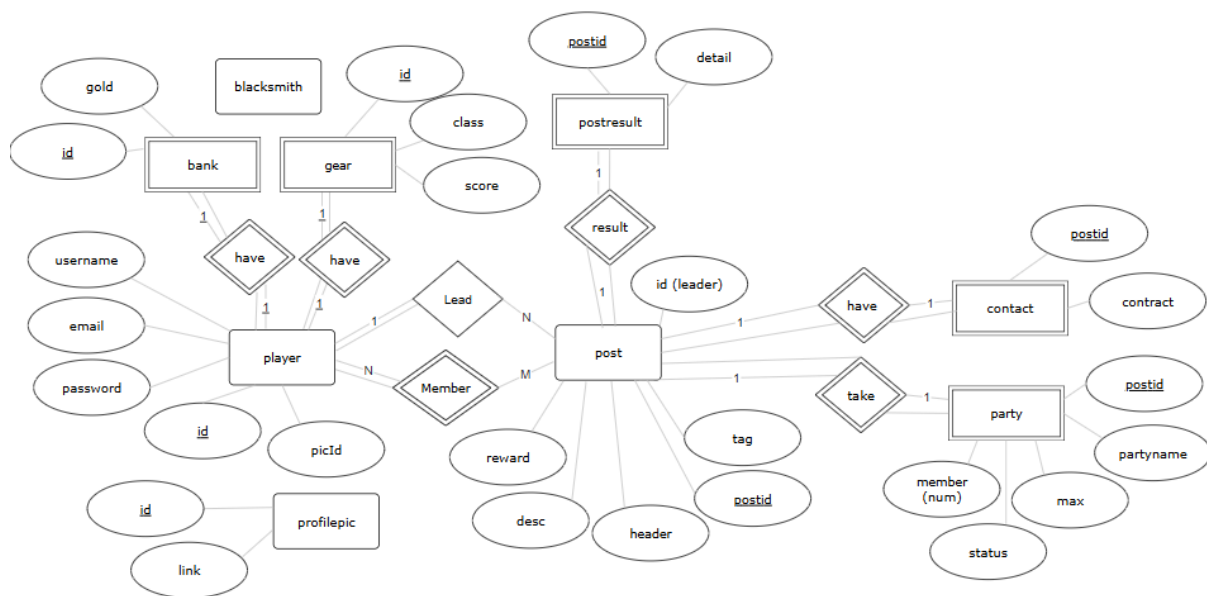
- party:
 - Primary Key: postId (INT) (ทำหน้าที่เป็น PK เพื่อระบุกลุ่มที่ผูกกับประกาศนั้นๆ) (1:1)
- member:
 - Foreign Keys: * id เชื่อมไปยัง post(id) (เพื่อระบุว่าเข้าปาร์ตี้ของโพสต์ไหน) และ playerId เชื่อมไปยัง player(id) (เพื่อระบุว่าใครคือสมาชิกในปาร์ตีดั้งนั้น) (M,N)

3.3.3. ตารางผลลัพธ์และการติดต่อ

- postresult:
 - Foreign Key: postId เชื่อมไปยัง post(id) เพื่อบันทึกผลลัพธ์ของภารกิจนั้นๆ (1:1)
- contact:
 - Foreign Key: postId เชื่อมไปยัง post(id) เพื่อเก็บข้อมูลการติดต่อสื่อสารที่เกี่ยวข้องกับภารกิจ (1:1)

3.3.4. ตารางระบบไอเทมและร้านตีเหล็ก

- gear:
 - Foreign Key: id (ภายในตาราง) เชื่อมไปยัง player(id) เพื่อระบุว่า gearscore นี้เป็นของผู้เล่นคนใด (1:1)
- blacksmith:
 - Primary Key : itemId ยืนยัน ของสวมใส่ ชิ้นนั้นๆ และ รายละเอียดของอุปกรณ์ชิ้นนั้นๆ



3.3.6 Normalization

เพื่อให้ฐานข้อมูลรองรับระบบการหากลุ่มที่มีความซับซ้อน ผู้จัดทำได้ดำเนินการตรวจสอบโครงสร้างตารางตามหลักการ Normalization ดังนี้:

1. First Normal Form (1NF)

- **หลักการ:** ทุก Attribute ในแต่ละ Tuple ต้องมีค่าเดียว (Atomic Value) และไม่มีกลุ่มข้อมูลที่ซ้ำซ้อนกัน (Repeating Groups)
- **การประยุกต์ใช้:** ในตาราง **member** เราไม่เก็บรายชื่อสมาชิกทุกคนไว้ใน Field เดียวของตาราง **party** แต่ใช้การแยกออกมาเป็นแถวข้อมูลเฉพาะ (Atomic rows) แทน

2. Second Normal Form (2NF)

- **หลักการ:** ต้องผ่านเงื่อนไข 1NF และทุก Non-key Attribute ต้องขึ้นตรงกับ Primary Key ทั้งหมด (Full Functional Dependency) โดยไม่มีการขึ้นตรงกับเพียงบางส่วนของคีย์ (Partial Dependency)
- **การประยุกต์ใช้:** ผู้จัดทำได้แยกข้อมูลที่ไม่เกี่ยวข้องออกจากกัน เช่น ข้อมูลรูปโปรไฟล์ ไม่ถูกเก็บไว้ในตาราง **player** โดยตรง แต่แยกไปไว้ในตาราง **profilepic** แล้วใช้ picid เชื่อมโยงมาแทน เพื่อป้องกัน

ไม่ให้ข้อมูล URL ของรูปภาพต้องขึ้นตรงกับ Primary Key ของผู้เล่นโดยตรง ซึ่งอาจทำให้เกิดความซ้ำซ้อนหากมีผู้เล่นหลายคนใช้รูปเดียวกัน

3. Third Normal Form (3NF)

- **หลักการ:** ต้องผ่านเงื่อนไข 2NF และต้องไม่มี Transitive Dependency (Non-key Attribute ต้องไม่ขึ้นตรงกับ Non-key Attribute ด้วยตัวเอง)
- **การประยุกต์ใช้:** * ในตาราง **gear** และ **blacksmith**: ข้อมูลรายละเอียดอุปกรณ์ (Gear) และการปรับแต่ง (Blacksmith) ถูกแยกออกจากตาราง **player** เพื่อให้ข้อมูล gear score ขึ้นตรงกับตาราง gear ของตัวเองเท่านั้น ไม่ได้ขึ้นตรงกับข้อมูลส่วนตัวของผู้เล่น
 - ในตาราง **bank**: ข้อมูลการเงินถูกแยกออกมาเป็นตารางเฉพาะ เพื่อให้ค่า gold ขึ้นตรงกับ id ของบัญชีธนาคารนั้นๆ เท่านั้น ไม่ไปปะปนกับข้อมูลในตาราง post หรือ party

3.4 ขั้นตอนการพัฒนา (Implementation)

1.Backend Development: (ในส่วนนี้คุณ sẽลงรายละเอียดเกี่ยวกับการสร้าง API ด้วย Node.js และ Express การจัดการ Routing และการเชื่อมต่อ SQL Query)

3.4.1 การพัฒนาส่วนประมวลผลหลังบ้าน (Backend Development)

ผู้จัดทำได้พัฒนาส่วน Backend ด้วย **Node.js** และ **Express Framework** โดยเน้นการสร้าง RESTful API เพื่อเชื่อมต่อกับฐานข้อมูล MySQL ผ่าน mysql2 package ซึ่งมีการจัดการส่วนสำคัญดังนี้:

3.4.1. การจัดการเส้นทางและตรรกะทางธุรกิจ (Routing & Business Logic)

ระบบมีการรับ-ส่งข้อมูลผ่าน HTTP Method ต่าง ๆ โดยใช้ระบบ **Cookies** ในการเก็บ Identity ของผู้เล่น (Player ID) เพื่อความปลอดภัยและความสะดวกในการใช้งาน โดยมีคำสั่ง Query ที่สำคัญในแต่ละส่วนดังนี้:

- **ระบบกระดานประกาศ (Main Board):** ในการดึงข้อมูลการกีฬา มาแสดง ระบบจะกรองเฉพาะโพสต์ที่ผู้ใช้ "ยังไม่ได้เข้าร่วม" และต้องมีสถานะเป็น "ACTIVE" เท่านั้น โดยใช้เทคนิคการ Join 3 ตารางและการทำ Subquery เพื่อตรวจสอบความซ้ำซ้อนของสมาชิก:

```
SELECT post.*, player.username  
FROM post
```

```

JOIN player ON player.id = post.id
JOIN party ON party.postId = post.postid
WHERE (post.postid NOT IN (
    SELECT postid FROM post
    JOIN member ON member.id = post.postid
    WHERE member.playerId = (?)
)) AND party.status = "ACTIVE";

```

- **ระบบค้นหาด้วยแท็ก (Search by Tag):** ใช้คำสั่ง LIKE ร่วมกับข้อมูลที่ได้รับมาจาก Request Body เพื่อให้ให้นักพัฒนาระบบสามารถค้นหาภารกิจตามประเภทที่สนใจได้:

```

SELECT post.*, party.partyname, player.username, party.status
FROM post
JOIN party ON post.postid = party.postId
JOIN player ON post.id = player.id
WHERE post.tag LIKE "{$req.body.tag}%";

```

3.4.2. การจัดการธุรกรรมและข้อมูลที่เกี่ยวข้อง (Transaction Logic)

เนื่องจากหนึ่งกิจกรรมของผู้ใช้ส่งผลกระทบต่อหลายตาราง ระบบจึงต้องจัดการ Query แบบลำดับขั้นตอน (Sequence) เพื่อรักษาความถูกต้องของข้อมูล:

- **การสร้างภารกิจใหม่ (Post Bounty):** เมื่อผู้เล่นสร้างโพสต์ ระบบจะทำการ Insert ข้อมูลลงใน 4 ตารางที่เกี่ยวข้องกันตามลำดับ คือ post (เพื่อรับ postid), contact, party, และ member เพื่อให้ผู้สร้างโพสต์เข้าเป็นสมาชิกคนแรกของปาร์ตี้โดยอัตโนมัติ

```

INSERT INTO post (`id`, `header`, `desc`, `reward`, `tag`) VALUE (?)

SELECT postid FROM post WHERE id = (?) ORDER BY postid desc"
INSERT INTO contact (`postid`, `contact`) VALUE (?)
INSERT INTO party (`postid`, `partyname`, `max`) VALUE (?)
INSERT INTO member (`id`, `playerId`) VALUE (?)

```

- **การเข้าร่วมกลุ่ม (Sign up Party):** ระบบจะทำการตรวจสอบเงื่อนไขก่อนเสมอ โดยดึงจำนวนสมาชิกปัจจุบัน (member) เทียบกับจำนวนที่รับได้สูงสุด (max) หากปาร์ตี้ยังไม่เต็ม ระบบจึงจะดำเนินการ UPDATE เพิ่มจำนวนสมาชิกและ INSERT ข้อมูลลงในตาราง member

"UPDATE party SET member = (?) WHERE (postId = (?))" add member by 1

"INSERT INTO member (`id`, `playerId`) VALUE (?)"

- **การออกจากกลุ่ม (Leave Party):** เป็นการย้อนกระบวนการ โดยระบบจะลดจำนวนสมาชิกในตาราง party และทำการลบแถวข้อมูลในตาราง member ที่ตรงกับรหัสผู้เล่นและรหัสโพสต์นั้นๆ:

UPDATE party SET member = (?) WHERE postId = (?);

DELETE FROM member WHERE playerId = (?) AND id = (?);

3. การจัดการความปลอดภัย (Security & Optimization)

- **Parameterized Queries:** ใช้เครื่องหมาย ? (Placeholders) ในการรับค่าจาก User เพื่อป้องกันการโจมตีแบบ SQL Injection ตามมาตรฐานความปลอดภัย
- **Relational Mapping:** การใช้ JOIN แทนการ Query หลายรอบ ช่วยลด Load ของ Database Server และทำให้การแสดงผลหน้า Frontend (React) มีความรวดเร็ว

```

1 import { db } from "../connect.js";
2
3 // getPost send back every post that are not posted or been a member of and Party that is active
4 export const getPost = (req,res) =>{
5   const q = `SELECT post.*,player.username FROM post JOIN player ON player.id = post.id JOIN party ON party.postId = post.postid WHERE post.id = ${req.body.id}`
6   db.query(q,[req.body.id] ,(err,data)=>{
7     if(err) return res.json(err)
8     return res.json(data)
9   })
10 }
11 // the opposite send back post (party) that you are in
12 export const getParty = (req,res) =>{
13   const q = `SELECT post.postid, member.playerId AS member, partyname, party.member AS num, party.max, party.status FROM post JOIN party ON party.postId = post.postid WHERE party.member = ${req.body.id}`
14   db.query(q,[req.body.id] ,(err,data)=>{
15     if(err) return res.json(err)
16     return res.json(data)
17   })
18 }
19 // send insert into post to db
20 export const postBounty = (req,res) =>{
21   const q = `INSERT INTO post ('id', 'header', 'desc', 'reward', 'tag') VALUE (?)`
22   const value = [
23     req.body.id,
24     req.body.header,
25     req.body.desc,
26     req.body.reward,
27     req.body.tag
28   ]
29   db.query(q,[value] ,(err,data)=>{
30     if(err) return res.json(err)
31     //return res.status(200).json("Created")
32   })
33 }
34 // find the lasted postid of this user
35 const q2 = `SELECT postid FROM post WHERE id = (?) ORDER BY postid desc`

```

2.Frontend Development: * ใช้ React ในการสร้าง Component ที่ทำงานแยกส่วนกัน (Reusable Components)

- ใช้ Tailwind CSS ในการสร้างสไตล์แบบ Custom เพื่อคุมโทนสีน้ำตาล-ทอง และพื้นผิวลายไม้ ตามที่ออกแบบไว้

```

313 <main className="flex-1">
314   {/* Reduced gap from 8 to 4 for a denser board feel */}
315   <div className="grid grid-cols-1 lg:grid-cols-2 xl:grid-cols-3 gap-4">
316     {posts.map((quest) => (
317       <div
318         key={quest.postid}
319         className="group relative p-5 shadow-[8px_8px_15px_rgba(0,0,0,0.3)] hover:shadow-[12px_12px_20px_rgba(0,0,0,0.4)] transition"
320         style={{
321           backgroundImage: 'url("https://www.transparenttextures.com/patterns/paper-fibers.png")',
322           backgroundColor: 'rgba(253, 246, 227, 0.9)',
323           backdropFilter: 'blur(4px)'
324         }}
325       >
326         {/* The Wax Seal/Pin */}
327         <div className="absolute top-2 left-1/2 -translate-x-1/2 w-3 h-3 bg-red-800/90 rounded-full shadow-inner border-2 border-black">
328
329           {/* Small background insignia */}
330           <div className="absolute -right-2 -top-2 opacity-5 text-[#3e2723]">
331             <Shield size={100} />
332           </div>
333
334           {/* TOP BAR: Tighter margins and smaller text */}
335           <div className="flex justify-between items-center mb-3 border-b border-dashed border-[#d7ccc8] pb-2">
336             <span className="text-[17px] font-black bg-[#5d4037] text-[#f5f5f5] px-2 py-0.5 rounded tracking-tighter uppercase">
337               {quest.tag}
338             </span>
339             <span className="text-lg font-black text-[#5d4037]">
340               {quest.reward}
341             </span>
342           </div>
343
344           {/* HEADER: Reduced size and min height */}

```

- เชื่อมต่อข้อมูลจาก Backend ผ่านการเรียกใช้ API เพื่อแสดงผลข้อมูลแบบ Dynamic

```
// getting post from backend
const [posts,setPost] = useState([]);
// get post that are not posted by user or member of
useEffect(()=>{
  const FetchAllPost = async ()=>{
    try{
      const res = await axios.post("http://localhost:8800/api/posts/getPost",{
        id:id
      })
      setPost(res.data);
    }catch(err){
      console.log(err)
    }
  }
  FetchAllPost()
},[id])
//console.log(posts)
```

3.5 การทดสอบส่วนหลังบ้าน (Backend Testing)

ในการทดสอบประสิทธิภาพและความถูกต้องของ API ผู้จัดทำเลือกใช้เครื่องมือ **Insomnia REST Client** เพื่อจำลองการส่ง Request (POST, GET) ไปยัง Server และตรวจสอบผลลัพธ์ที่ตอบกลับมาจากฐานข้อมูล MySQL โดยมีรายละเอียดขั้นตอนการทดสอบดังนี้:

3.5.1 การทดสอบการเข้าถึงข้อมูล (Endpoint Testing)

ผู้จัดทำได้สร้างชุดการทดสอบ (Request Collection) เพื่อตรวจสอบความถูกต้องของ Logic ในส่วนต่างๆ ดังนี้:

- **Authentication Test:** ทดสอบการสมัครสมาชิก (Register) และการเข้าสู่ระบบ (Login) เพื่อตรวจสอบว่ารหัสผ่านถูกจัดการอย่างปลอดภัยและระบบสามารถออก Token หรือ Session ได้ถูกต้อง
- **Bounty & Party CRUD Test:** ทดสอบการสร้าง (Create), อ่าน (Read), แก้ไข (Update) และลบ (Delete) ข้อมูลในตาราง post และ party เพื่อให้มั่นใจว่าข้อมูลนักพัฒนาและภารกิจถูกบันทึกลงฐานข้อมูลอย่างแม่นยำ
- **Membership Logic Test:** ทดสอบการส่ง Request เพื่อเข้าร่วมกลุ่ม (Join Party) และตรวจสอบว่าตาราง member มีการบันทึกความสัมพันธ์ระหว่าง player_id และ party_id ได้ถูกต้องตามเงื่อนไขที่กำหนด

3.5.2 การตรวจสอบความถูกต้องของรูปแบบข้อมูล (Data Validation)

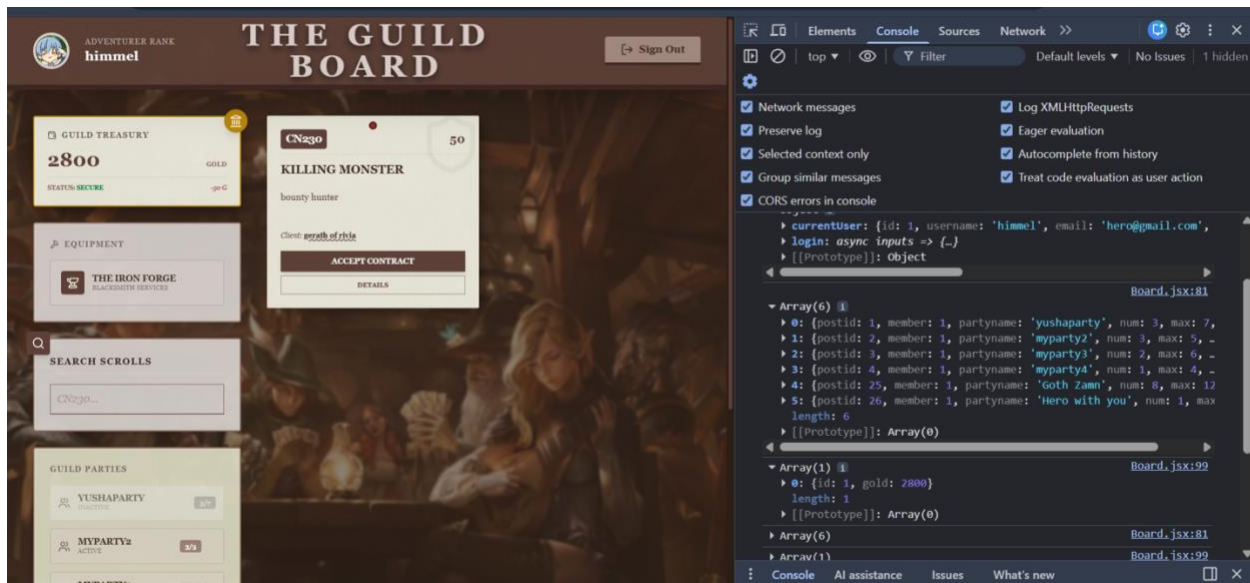
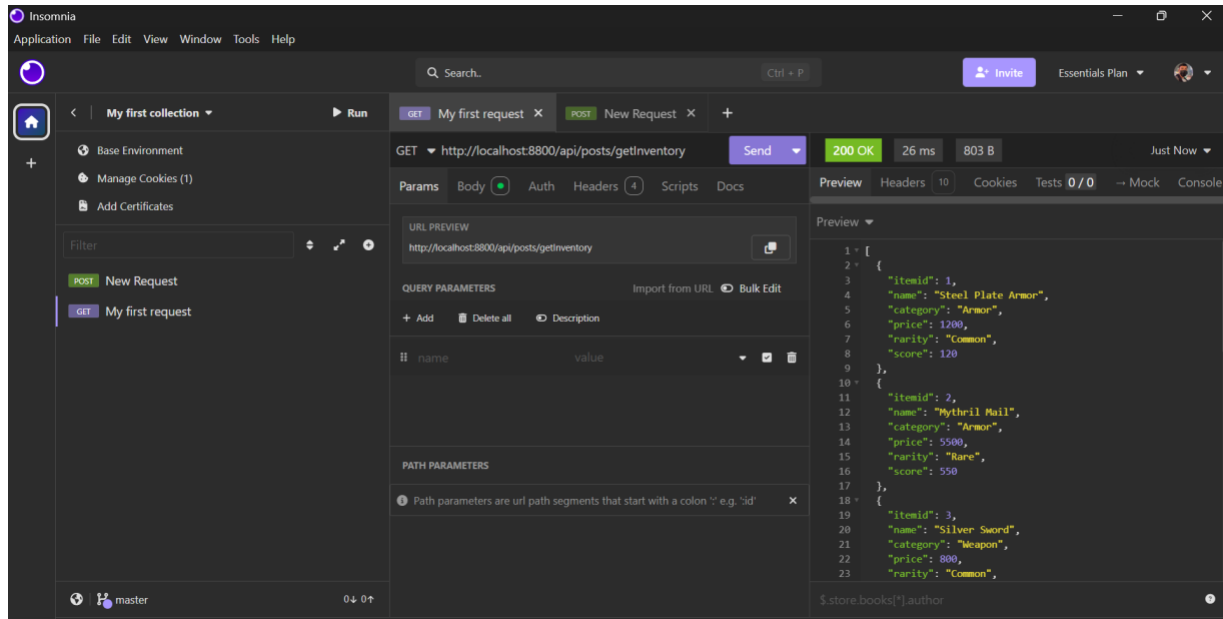
- **Status Code Verification:** ตรวจสอบ HTTP Status Code ที่ตอบกลับมา เช่น 200 OK เมื่อดำเนินการสำเร็จ, 400 Bad Request เมื่อข้อมูลที่ส่งมาไม่ครบถ้วน หรือ 401 Unauthorized เมื่อไม่มีสิทธิ์เข้าถึงข้อมูล
- **JSON Response Structure:** ตรวจสอบโครงสร้างข้อมูล JSON ที่ API ส่งกลับมามีฟิลด์ข้อมูล (Fields) ตรงตามที่ Frontend (React) ต้องการ เพื่อลดความผิดพลาดในการแสดงผล

3.5.3 การทดสอบความเชื่อมโยงของฐานข้อมูล (Database Integration Test)

ในระหว่างการส่ง Request ผ่าน Insomnia ผู้จัดทำจะทำการตรวจสอบควบคู่ไปกับ MySQL Workbench เพื่อยืนยันว่า:

- การเพิ่มข้อมูลในตารางหนึ่ง ไม่ทำให้ข้อมูลในตารางที่มีความสัมพันธ์กัน (Foreign Key) เกิดความเสียหาย

- ระบบสามารถรองรับการ Query ข้อมูลที่มีความซับซ้อน (Complex Joins) เช่น การดึงรายชื่อสมาชิกทั้งหมดในปาร์ตี้ออกมาแสดงผลผ่าน API ได้อย่างรวดเร็ว



บทที่ 4

ผลการดำเนินงาน

จากการพัฒนาระบบเว็บแอปพลิเคชันสำหรับการบริหารจัดการและจับคู่สมาชิกกลุ่ม ผู้จัดทำสามารถสรุปผลการดำเนินงานแบ่งตามส่วนประกอบหลักของระบบได้ดังนี้

4.1 ผลการพัฒนาส่วนติดต่อผู้ใช้งาน (Frontend Result)

ส่วนติดต่อผู้ใช้งานได้รับการออกแบบในธีม **Fantasy Wooden & Parchment** เพื่อสร้างบรรยากาศการใช้งานที่ผ่อนคลายและน่าสนใจ โดยผลลัพธ์ในแต่ละหน้าจอหลักมีรายละเอียดดังนี้:

1. **หน้าระบบพิสูจน์ตัวตน (Login & Register):** ผู้ใช้งานสามารถลงทะเบียนเข้ากิลด์ (สมัครสมาชิก) และเข้าสู่ระบบได้ โดยระบบจะทำการเก็บข้อมูลตัวตนผ่าน Cookies เพื่อใช้ในการทำกิจกรรมต่าง ๆ ภายในเว็บไซต์
2. **หน้ากระดานภารกิจ (The Main Board):** แสดงรายการประกาศหากลุ่ม (Bounty) ในรูปแบบของการ์ดกระดาษเก่า โดยจะแสดงผลเฉพาะกลุ่มที่ยังมีสถานะ "ACTIVE" และเป็นกลุ่มที่ผู้เล่นยังไม่ได้เข้าร่วมเท่านั้น เพื่อลดความสับสนในการเลือกกลุ่ม
3. **หน้าสร้างภารกิจ (Post Bounty):** ผู้ใช้งานสามารถสร้างประกาศใหม่ได้ โดยระบุหัวข้อ (Header), รายละเอียด (Description), รางวัล (Reward), และแท็ก (Tag) เพื่อระบุประเภทของงานกลุ่ม ซึ่งข้อมูลจะถูกกระจายไปบันทึกลงในตารางที่เกี่ยวข้องทันที
4. **หน้าระบบจัดการปาร์ตี้ (Party Management):** แสดงรายชื่อสมาชิกในทีมและจำนวนที่เหลืว่างอยู่ ระบบสามารถตรวจสอบและป้องกันการกดเข้าร่วมหากปาร์ตี้เต็ม (Full) ได้โดยอัตโนมัติ

4.2 ผลการทำงานส่วนประมวลผลหลังบ้าน (Backend Result)

จากการทดสอบระบบผ่าน **Insomnia** และการใช้งานจริงร่วมกับ Frontend พบว่าส่วน Backend สามารถจัดการ Logic ที่ซับซ้อนได้ดังนี้:

- **Relational Query Performance:** การใช้คำสั่ง JOIN ระหว่างตาราง post, player, และ party ช่วยให้การเรียกแสดงผลข้อมูลบนหน้ากระดานหลักมีความถูกต้องและแสดงชื่อผู้โพสต์ได้ทันที
- **Membership Validation:** ระบบสามารถตรวจสอบความซ้ำซ้อนของสมาชิกได้อย่างแม่นยำ (ผ่าน Subquery) โดยป้องกันไม่ให้ผู้เล่นเห็นกลุ่มที่ตนเองเป็นสมาชิกอยู่แล้วในหน้ากระดานหลัก

- **Data Integrity:** เมื่อมีการสร้างโพสต์หรือการเข้าร่วมกลุ่ม ข้อมูลในตาราง member และ party จะถูกปรับปรุงให้สอดคล้องกันเสมอ (Consistent) ตามหลักการฐานข้อมูลเชิงสัมพันธ์

4.3 การแก้ปัญหาความเสียดทานในการหากลุ่ม (Problem Solving)

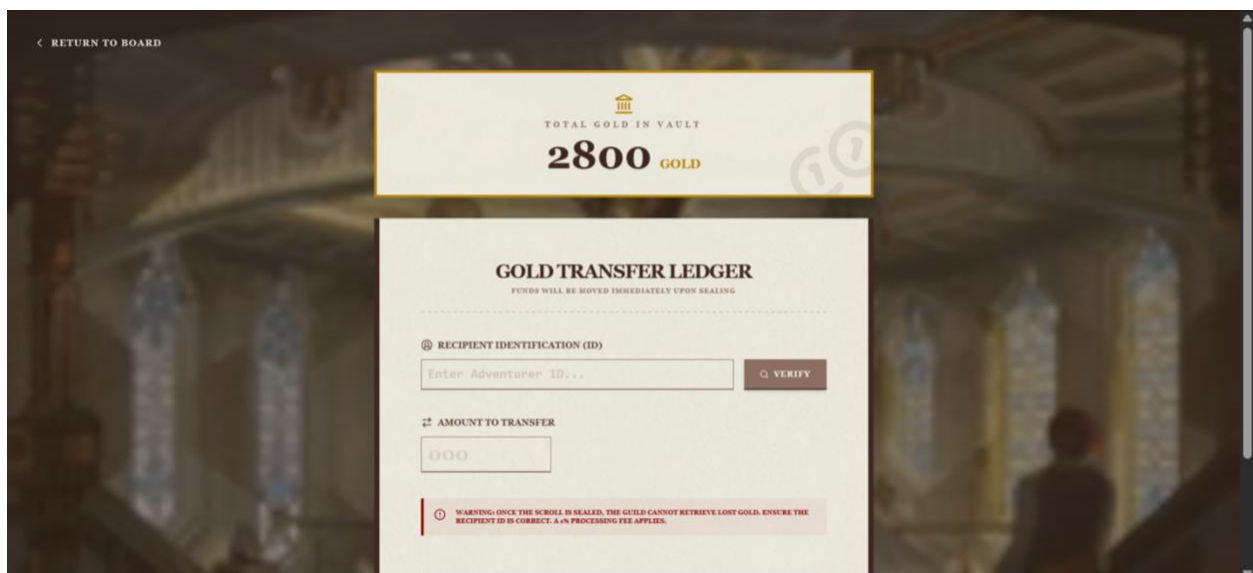
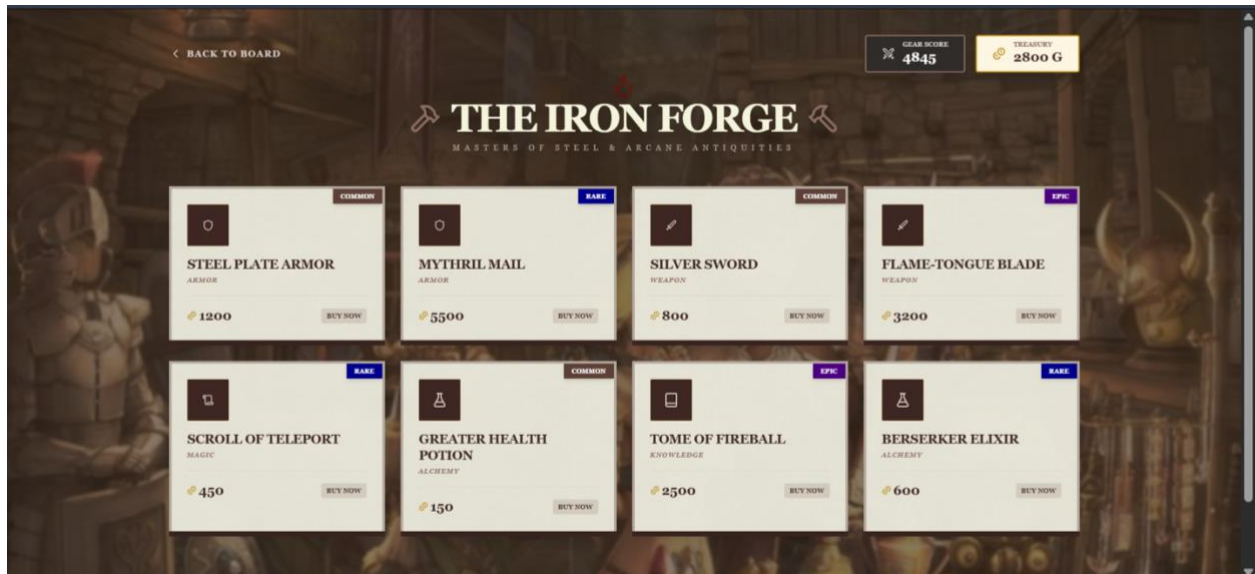
จากวัตถุประสงค์ที่ต้องการลดความเสียดทาน (Friction) ในการหาทำงานกลุ่ม ระบบสามารถทำหน้าที่เป็นสื่อกลางได้อย่างมีประสิทธิภาพดังนี้:

1. **Transparency:** ผู้ใช้เห็นข้อมูลชัดเจนว่ากลุ่มใดต้องการสมาชิกเพิ่มกี่คน และเป็นงานประเภทใด (ผ่านระบบ Tag)
2. **Speed:** ลดขั้นตอนการสอบถามส่วนตัวลง โดยผู้ใช้สามารถกด "Join Party" ได้ทันทีหากมีที่ว่าง
3. **Centralization:** ข้อมูลการติดต่อ (Contact) และรายละเอียดงานถูกรวบรวมไว้ที่จุดเดียว ไม่กระจัดกระจายเหมือนการประกาศผ่านโซเชียลมีเดียทั่วไป

4.4 ผลการทดสอบความถูกต้องของฐานข้อมูล

จากการตรวจสอบผ่าน MySQL Workbench พบว่าโครงสร้าง EER Diagram ที่ออกแบบไว้สามารถรองรับการจัดเก็บข้อมูลได้จริง โดยที่:

- คีย์หลัก (Primary Key) และคีย์นอก (Foreign Key) ทำหน้าที่เชื่อมโยงข้อมูลระหว่างตาราง player, post, และ member ได้อย่างไม่ผิดพลาด
- ระบบรองรับการเพิ่มขึ้นของข้อมูลนักผจญภัยและไอเทมในตาราง gear และ blacksmith โดยไม่กระทบต่อประสิทธิภาพการทำงานของระบบหลัก



[RETURN TO PARTY HALL](#)



OFFICIAL COMPLETION FORM

DRINKING

Contract: party at tavern

 FIELD REPORT & PROOF OF COMPLETION

DESCRIBE THE OUTCOME AND PROVIDE LINKS TO EVIDENCE
